Сортировки

Оливера Захаров Х. А.
ФГБОУ ВО «МИРЭА - Российский технологический университет»
зима-весна 2026 г.

РТУ МИРЭА
Институт искусственного интеллекта

Сортировка пузырьком (Bubble sort)

Демонстрация

6 5 3 1 8 7 2 4

РТУ МИРЭА

Институт искусственного интеллекта

Сортировка пузырьком (Bubble sort). Реализация

```
def bubblesort(listt):
    n = len(listt)
    for i in range(n): # проверка перестановок
        swapped = False
        for j in range(0, n-i-1): # реализация перестановок
            if listt [j] > listt [j+1]:
                listt[j], listt[j+1] = listt [j+1], listt[j]
                swapped = True
        if (swapped == False): # проверка перестановок
            break
```

1) Сравнивается текущий элемент со следующим
2) Происходит перестановка элементов при условии , что следующий элемент больше текущего
3) Как только проходит цикл по всему списку, последний элемент закрепляется и не учитывается при следующем проходе, т.к. он заведомо будет самым большим
4) алгоритм повторяется для элемента *i+1*

Алгоритм с каждым новым проходом как бы выталкивает наибольший элемент «на поверхность», как пузырёк кислорода, который стремится оказаться наверху!

РТУ МИРЭА
Институт искусственного интеллекта

Сортировка вставкой (Insertion sort)

Демонстрация

6 5 3 1 8 7 2 4

РТУ МИРЭА

Институт искусственного интеллекта

Сортировка вставкой (Insertion sort). Реализация

```
def insertionsort(listt):
    n = len(listt)
    if n <= 1:
        return
    for i in range(1, n): # проход по каждому элементу
        key = listt[i] # элемент последовательности
        j = i - 1
        while j >= 0 and key < listt[j]: # поиск элемента больше key
            listt[j + 1] = listt[j]
            j -= 1
        listt[j + 1] = key
```

1) Текущий элемент сравнивается последовательно со всеми предыдущими
2) Если текущий элемент меньше предыдущего, то происходит смещение на 1 элемент вправо
3) Как только попадается элемент, меньший текущего, элемент встаёт на позицию после меньшего элемента
4) Процесс повторяется до конца списка

РТУ МИРЭА
Институт искусственного интеллекта

Сортировка выбором (Selection sort)

Демонстрация

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

РТУ МИРЭА

Институт искусственного интеллекта

Сортировка выбором (Selection sort). Реализация

```
def selectionsort(listt, size):
    for ind in range(size - 1):
        min_index = ind
        for j in range(ind + 1, size):
            if listt [j] < listt [min_index]:
                min_index = j
        listt[ind], listt[min_index] = listt
        [min_index], listt[ind]
```

1) Сохраняется позиция текущего индекса
2) Происходит сравнение текущего элемента с последующим
3) Как только встречается элемент, меньший текущего, сохраняется позиция меньшего элемента
4) При нахождении минимального элемента происходит перестановка минимального элемента и текущего
5) Процесс повторяется до конца списка

РТУ МИРЭА
Институт искусственного интеллекта

Сортировка слиянием (Merge sort)

Демонстрация

6 5 3 1 8 7 2 4

РТУ МИРЭА

Институт искусственного интеллекта

Сортировка слиянием (Merge sort). Реализация

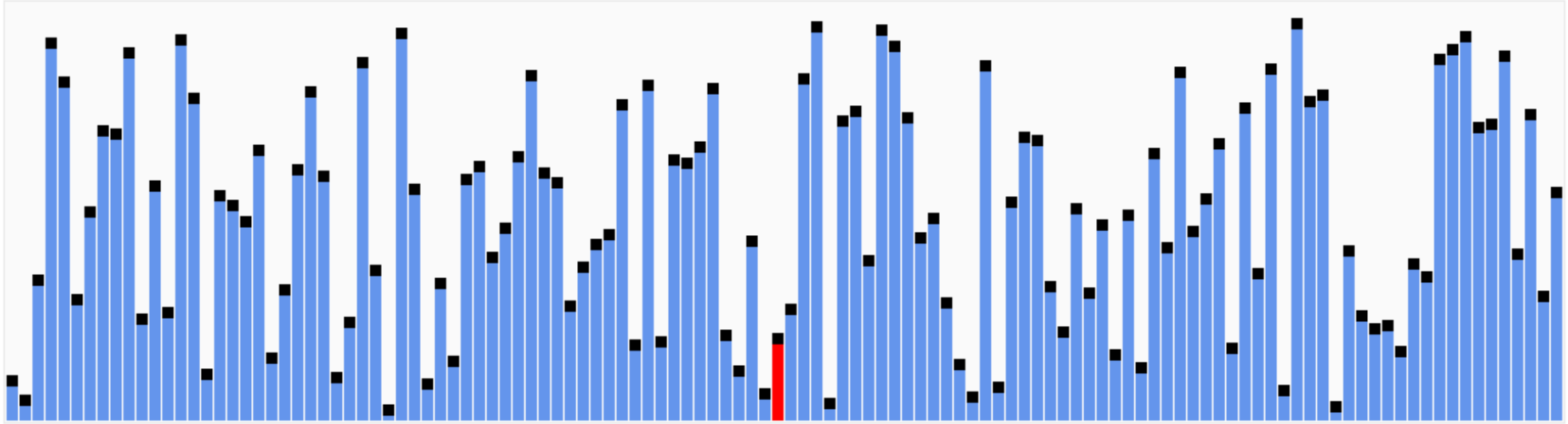
```
def merge_sort(listt):
    if len(listt) <= 1:
        return listt
    mid = len(listt) // 2 # массив делится на 2 части
    left = merge_sort(listt[:mid])
    right = merge_sort(listt[mid:])
    return merge(left, right)

def merge(left, right):
    sorted_list = []
    i = j = 0
    while i < len(left) and j < len(right): # сравнение элементов левой и правой части
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1
    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])
    return sorted_list
```

1) Массив делится на 2 части (левую и правую) если число нечётное, то длина одного из списков на 1 больше
2) Деление происходит до того момента, пока длина списка не станет равна 1 элементу
3) Происходит поэлементное сравнение двух списков
   a. сравниваются i-й элемент левого и j-й правого списка (на начальном этапе i=j)
   b. если элемент одного из списков меньше другого, то он добавляется во временный список и увеличивается индекс i+1 (или j+1)
   c. процесс происходит до тех пор, пока итерации не пройдут во всем элементам обоих списков
4) Процесс повторяется до тех пор пока не произойдёт слияние всех частей списка

РТУ МИРЭА
Институт искусственного интеллекта

Быстрая сортировка (Quick sort)

Демонстрация



РТУ МИРЭА

Институт искусственного интеллекта

Быстрая сортировка (Quick sort). Реализация

```
def quick_sort(listt):
    if len(listt) <= 1:
        return listt
    pivot = listt [len(listt) // 2] # pivot – опорный элемент
    left = [x for x in listt if x < pivot]
    middle = [x for x in listt if x == pivot]
    right = [x for x in listt if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

1) Выбирается опорный элемент
   a. случайное числа из списка
   b. 1-ый, последний или срединный элемент
   c. и т.д.
2) Разбиение списка три части:
   - левая (все элементы меньше опорного)
   - правая (все элементы больше опорного)
   - срединные (все элементы равные опорному)
3) Алгоритм повторяется для левого и правого списка
4) После чего все эти части объединяются в единый список

РТУ МИРЭА
Институт искусственного интеллекта

Сравнительный анализ алгоритмов

Сортировки	Сложность средняя худшая	Память	Примечание
Пузырьковая	$O(n^2)$ $O(n^2)$	*1*	1. Эффективен на частично-отсортированных списках любой длины
Вставкой	$O(n^2)$ $O(n^2)$	*1*	1. Эффективен на небольших списках 2. Эффективен в основном на частично-отсортированных
Выбором	$O(n^2)$ $O(n^2)$	*1*	Отличается простотой реализации
Слиянием	$O(n \cdot \log n)$ $O(n \cdot \log n)$	*n*	Хорошо масштабируется для очень больших списков
Быстрая	$O(n \cdot \log n)$ $O(n^2)$	$\log n$	На практике относится к числу самых быстрых алгоритмов сортировки

Институт искусственного интеллекта

Встроенные функции сортировки

Синтаксис

iterable.sort([, key, reverse])

print(iterable)

Синтаксис

new_iter = sorted(iterable, [, key, reverse])

print(new_iter)

iterable – итерируемый объект # *iterable = list() – в случае метода .sort()*

key = func – по умолчанию None # *функция, принимающая в качестве аргумента элемент последовательности*

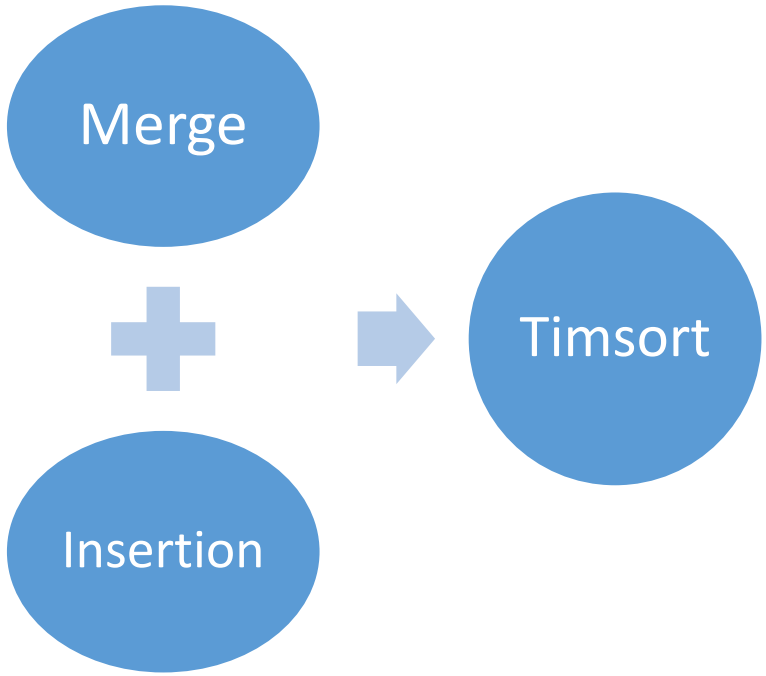
reverse = bool() – по умолчанию False # *True – в обратном порядке*

РТУ МИРЭА

Институт искусственного интеллекта

Алгоритм Timsort. Powersort 3.11+

Природа метода



Порядок действий

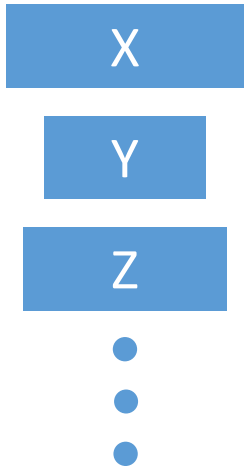
1. Находим минимальную оптимальную длину подсписка *minrun*
2. Ищем подсписок *run*, отсортированный либо нестрого по возрастанию, либо строго по убыванию (переставив элементы)
3. Если список *run* меньше *minrun*, то берём следующий подсписок и удлиняем сортировкой вставками и кладём в стек
4. Этап слияния

Этап слияния

Если:

$X > Y + Z \; and \; Y > Z$

то слияния не будет, в противном случае Y сливается с меньшим из X и Z



РТУ МИРЭА
Институт искусственного интеллекта

Сортировка подсчётом

13	5	8	2	1	9	8	5	7	3

исходный список

0	0	0	0	0	0	0	0	0	0	0	0	0

список из нулей (длина – значение максимального элемента)

0	1	1	1	0	2	0	1	2	1	0	0	0	1

заполненный список (i – значение элемента из списка, значение по i – число элементов в списке)

1	2	3	5	5	7	8	8	9	13

отсортированный список

РТУ МИРЭА

Институт искусственного интеллекта

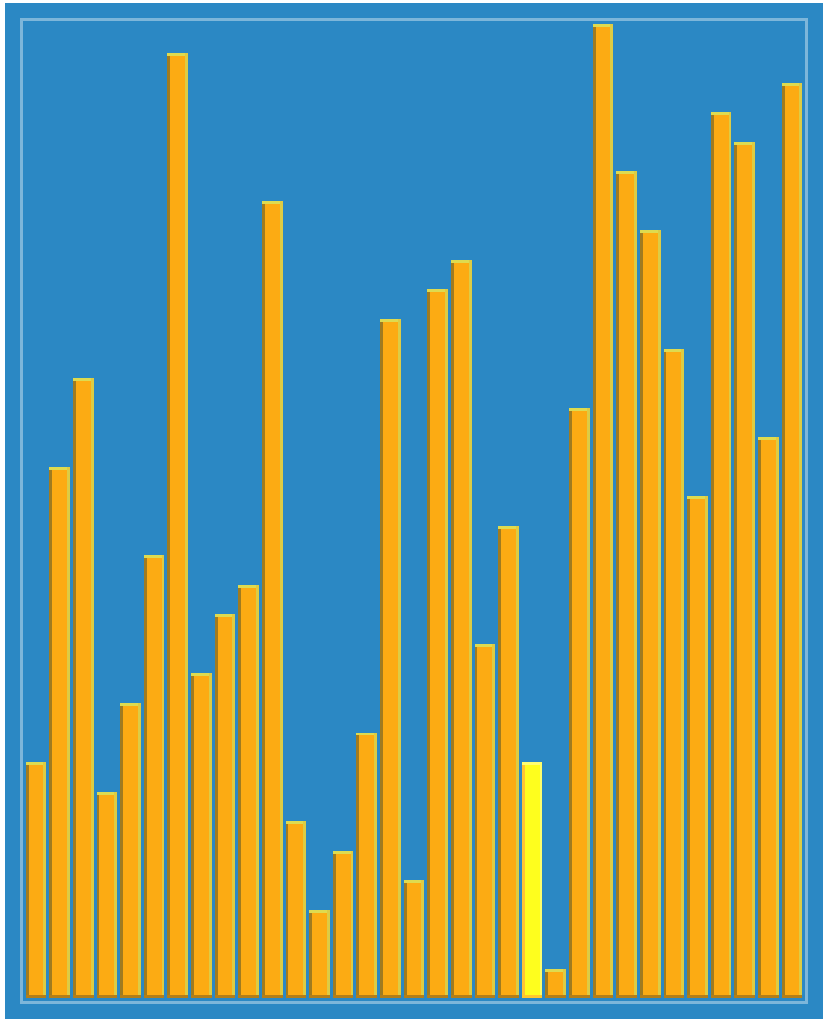
Сортировка подсчётом. Реализация

```
def countersort(num_list):
    max_elem = -1
    for elem in num_list:
        if elem > max_elem:
            max_elem = elem
    temp_list = [0 for _ in range(max_elem + 1)]
    for elem in num_list:
        temp_list[elem] = temp_list[elem] + 1
    for i in range(len(temp_list)):
        while temp_list[i] != 0:
            num_list[j] = i
            j = j + 1
            temp_list[i] = temp_list[i] - 1
```

1) Происходит поиск максимального по значению элемента последовательности
2) Формируется список (длина – максимальное значение элемента последовательности), заполняющийся нулями
3) Новый список заполняется значениями, где:
    a. индекс списка (i) – значение элемента
    b. значение по индексу – то, сколько раз встретился элемент (i) в исходной последовательности
4) Исходный список заполняется новыми значениями в соответствии с новым списком

РТУ МИРЭА
Институт искусственного интеллекта

Сортировка Шелла



РТУ МИРЭА

Институт искусственного интеллекта

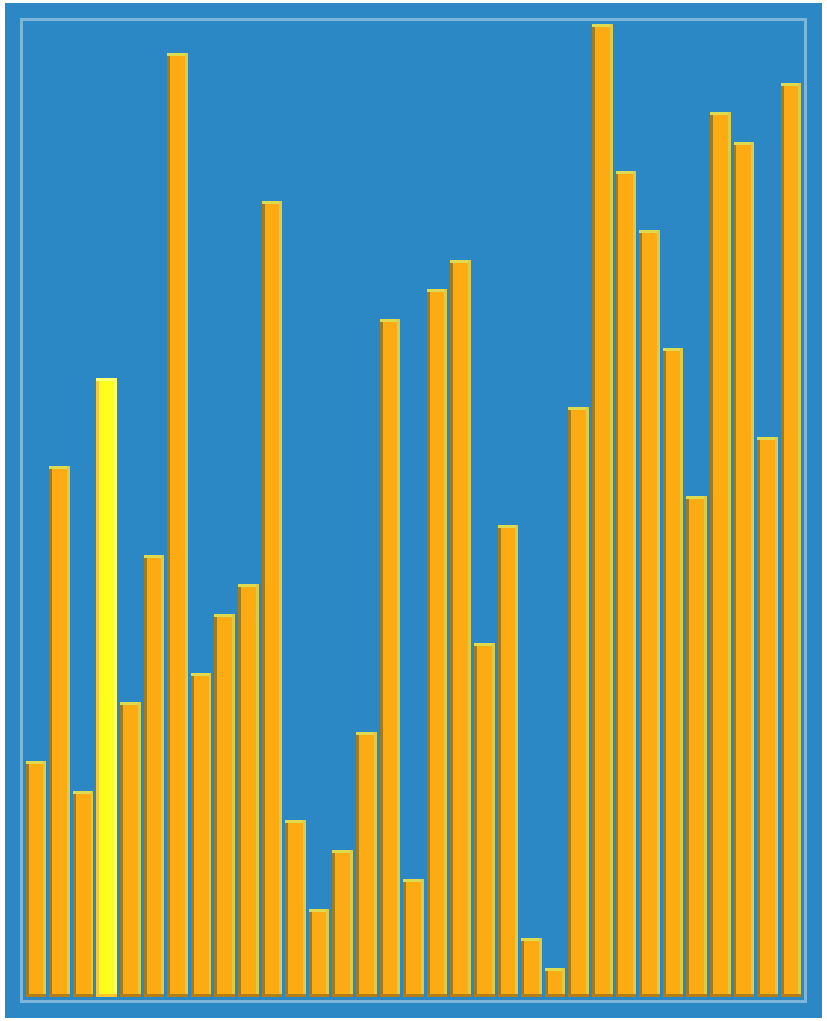
Сортировка Шелла. Реализация

```
def shellsort(num_list):
    d = len(num_list) // 2
    while d != 0:
        for i in range(d, len(num_list)):
            while num_list[i-d] > num_list[i]
            and i - d >= 0:
                num_list[i], num_list[i - d] =
                num_list[i - d], num_list[i]
                i = i – d
        d = d // 2
    return num_list, count-1
```

1) Формируем шаг, с которым будет происходить сортировка вставками
   a. $d = \frac{N}{2}, \dots, d_i = \frac{d_{i-1}}{2}$, где *N – len(num_list)* (Шелла)
   b. $d_i = 2^i - 1$ (Хиббарда)
   c. $d_i = 9 * 2^i - 9 * 2^{\frac{i}{2}} + 1$ if a%2 == 0
      $d_i = 8 * 2^i - 6 * 2^{\frac{(i+1)}{2}} + 1$ (Седжвика)
2) Берутся все элементы, стоящие друг от друга на расстоянии d и сортируются вставкой
3) После окончания сортировки берётся следующий набор элементов с шагом *i+1*
4) Операция продолжается до прохода по всем элементам списка и повторяется с новым шагом

РТУ МИРЭА
Институт искусственного интеллекта

Гномья сортировка



РТУ МИРЭА

Институт искусственного интеллекта

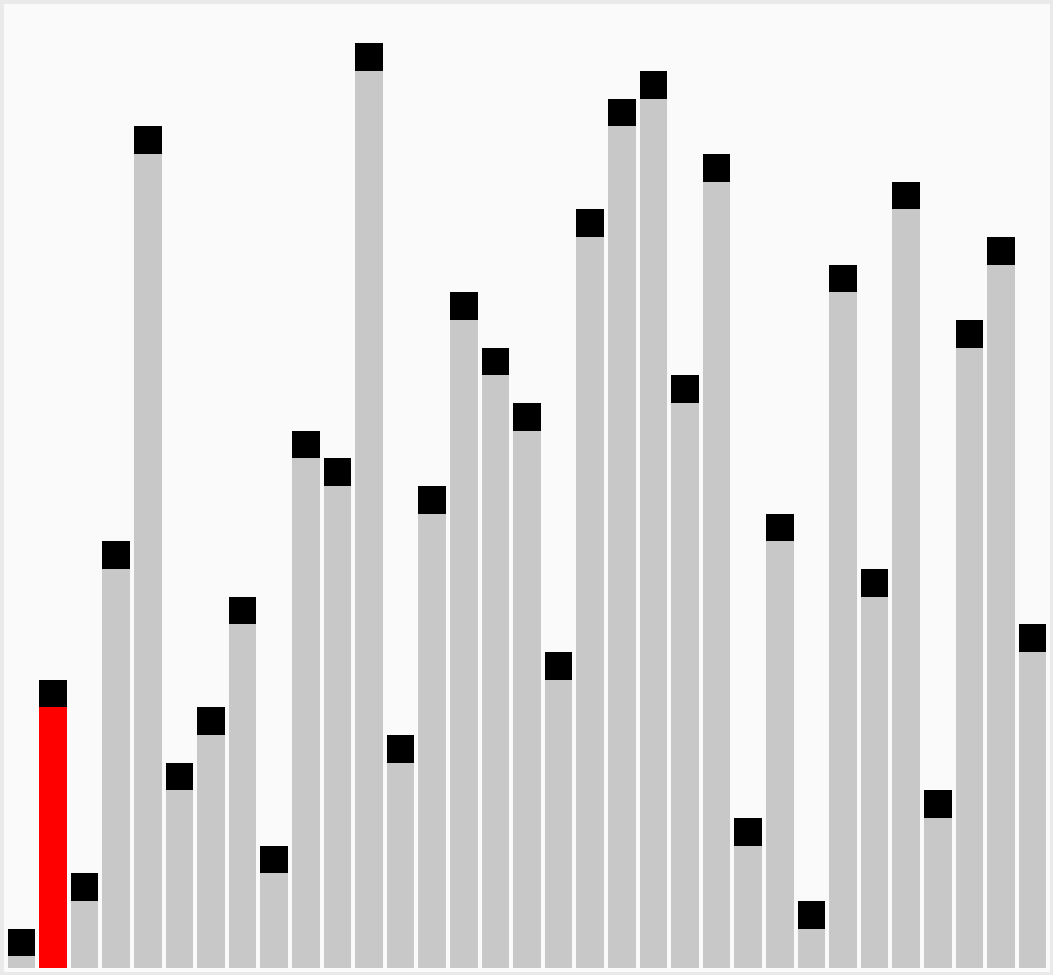
Гномья сортировка. Реализация

```
def gnomesort(num_list):
    i = 1
    j = 1
    while i < len(num_list):
        if num_list[j] < num_list[j-1]:
            num_list[j], num_list[j-1] =
            num_list[j-1], num_list[j]
            j = j - 1
            if j < 1:
                i = i + 1
                j = i
        else:
            i = i + 1
            j = i
```

1) Проход идёт с (i+1)-го элемента и идёт в обратную сторону
2) Происходит сравнение текущего элемента с предыдущим
3) Если предыдущий элемент больше, происходит обмен (перестановка)
4) При постановке текущего элемента в нужное место, операции повторяются для следующего элемента

РТУ МИРЭА
Институт искусственного интеллекта

Шейкерная сортировка



РТУ МИРЭА

Институт искусственного интеллекта

Шейкерная сортировка. Реализация

```
def cocktailsort(a):
    swapped = True
    end, start = len(a)-1, 0
    while (swapped == True):
        swapped = False
        for i in range(start, end):
            if (a[i] > a[i + 1]):
                a[i], a[i + 1] = a[i + 1], a[i]
                swapped = True
        if not swapped:
            break
        swapped = False
        end = end - 1
        for i in range(end-1, start-1, -1):
            if (a[i] > a[i + 1]):
                a[i], a[i + 1] = a[i + 1], a[i]
                swapped = True
        start = start + 1
```

1) Сравнивается текущий элемент со следующим
2) Происходит перестановка элементов при условии, что следующий элемент меньше текущего
3) Как только цикл проходит по всему списку, последний элемент закрепляется и не учитывается при следующем проходе, т.к. он заведомо будет самым большим
4) Теперь сортировка идёт в обратном порядке и «тянет на дно» самый маленький по значению элемент
    a. Сравнивается текущий элемент с предыдущим
    b. Происходит перестановка элементов при условии , что предыдущий элемент больше текущего
5) Операции продолжаются до тех пор пока число перестановок не будет = 0

РТУ МИРЭА
Институт искусственного интеллекта

Сравнительный анализ алгоритмов

Сортировки	Сложность средняя худшая	Память	Примечание
Подсчётом	$O(n+k)$ $O(n+k)$	$O(n+k)$	k – максимальное значение элемента ключа
Шелла	$O(n\log^2 n)$ $O(n^2)$	$O(1)$	Усовершенствованный вариант сортировки вставками, и среднее время зависит от выбранных шагов; не деградирует при неудачных наборах
Гномья	$O(n^2)$ $O(n^2)$	$O(1)$	Работает хорошо на почти отсортированных данных
Шейкерная	$O(n^2)$ $O(n^2)$	$O(1)$	Незначительно повышает производительность пузырьковой сортировки

Институт искусственного интеллекта

Задачи

1. У вас есть список случайных чисел случайно заданной длины *(задать длину и набор чисел можно через модуль random методом randint())*. Ваша задача заключается в том, чтобы написать программу, определяющую наибольший ***последовательно-идущий*** срез списка по нестрогому **>=** возрастанию и строгому **<** убыванию и сравнивающую оба этих варианта и выдающую наибольший из них срез вместе с указанием позиции среза ***(должна совпасть хотя бы длина)***

Пример1: [9, 16, 14, 3, 3, 1, 1, 18, 19, 8, 2, 15, 10, 2]

Вывод: Наибольший срез списка по возрастанию [1, 1, 18, 19] лежит в диапазоне от 5 до 8

Пример2: [19, 30, 7, 29, 13, 25, 20, 3, 17, 22, 20, 28, 19, 15, 6, 2, 2, 7, 12]

Вывод: Наибольший срез списка по по убыванию: [28, 19, 15, 6, 2] лежит в диапазоне от 11 до 15 позиции

РТУ МИРЭА

Институт искусственного интеллекта

Задачи

2. Вам необходимо попытаться реализовать концепт сортировки TimSort.

Одна из ключевых задач сортировки – сформировать список подсписков ***run*** исходной последовательности и объединить их до длины минимального значения (***minrun***) сортировкой вставками, получив стек (можно просто положить в новый список).

Поскольку по условию самой сортировки рекомендуемое значение размера минимального подсписка начинается от 32 до 65, то считайте минимальное значение ***minrun*** как степень числа 2.

Например, если у вас число 46, то minrun=6 т.к. 2^5 = 32 < 46, и 32 не покрывает всю длину списка: [[6 elems], …, [6 elems], [4 elems]]

Если 28 – minrun=5

Если 14 – minrun=4

РТУ МИРЭА

Институт искусственного интеллекта